

Agent Schema Loader

Agent Schema Loader

Overview

This document defines the **Agent Schema Loader** for your event-driven, Redis-backed, polyglot agent system. It provides a unified mechanism for loading, validating, and distributing agent schemas across:

- Gateway (validation of incoming tasks)
- Frontend (dynamic input form rendering)
- Agent Harness (input validation and runtime configuration)
- Workflow Engine (node configuration and capability mapping)

Agent schemas describe:

- Agent capabilities
- Input fields
- Output structure
- Required parameters
- Validation rules
- UI metadata

This loader ensures future-proof extensibility and multi-agent interoperability.

Folder Structure

```
/agent-schemas
/schemas
  exampleAgent.json
loader.ts
validator.ts
registry.ts

/gateway
/services
  agentSchemaService.ts

/frontend
/api
  fetchAgentSchema.ts
```

File: agent-schemas/loader.ts

```
import fs from "fs";
import path from "path";

export function loadSchema(agentId: string) {
  const filePath = path.join(__dirname, "schemas", `${agentId}.json`);

  if (!fs.existsSync(filePath)) {
    throw new Error(`Schema not found for agent: ${agentId}`);
  }

  const raw = fs.readFileSync(filePath, "utf-8");
  return JSON.parse(raw);
}
```

File: agent-schemas/validator.ts

```
export function validateInput(schema: any, input: any) {
  const errors: string[] = [];

  for (const field of schema.input) {
    const value = input[field.id];

    if (field.required && (value === undefined || value === null)) {
      errors.push(`Missing required field: ${field.id}`);
      continue;
    }

    if (field.type === "number" && typeof value !== "number") {
      errors.push(`Field ${field.id} must be a number`);
    }

    if (field.type === "string" && typeof value !== "string") {
      errors.push(`Field ${field.id} must be a string`);
    }
  }

  return errors;
}
```

File: agent-schemas/registry.ts

```
import { loadSchema } from "../loader";

const cache = new Map<string, any>();

export function getAgentSchema(agentId: string) {
  if (cache.has(agentId)) return cache.get(agentId);

  const schema = loadSchema(agentId);
  cache.set(agentId, schema);
  return schema;
}
```

File: gateway/services/agentSchemaService.ts

```
import { getAgentSchema } from "../../agent-schemas/registry";
import { validateInput } from "../../agent-schemas/validator";

export function validateTask(agentId: string, input: any) {
  const schema = getAgentSchema(agentId);
  const errors = validateInput(schema, input);

  if (errors.length > 0) {
    throw new Error(`Invalid input: ${errors.join(", ")}`);
  }

  return schema;
}
```

File: frontend/api/fetchAgentSchema.ts

```
export async function fetchAgentSchema(agentId: string) {
  const res = await fetch(`http://localhost:3001/agents/${agentId}/schema`);
  return await res.json();
}
```

Example Agent Schema: agent-schemas/schemas/exampleAgent.json

```
{
  "id": "exampleAgent",
  "name": "Example Agent",
  "input": [
    {
      "id": "query",
      "label": "Query",
      "type": "string",
      "required": true
    },
    {
      "id": "maxTokens",
      "label": "Max Tokens",
      "type": "number",
      "required": false,
      "default": 256
    }
  ],
  "output": {
    "type": "string"
  }
}
```

Integration Notes

Gateway Responsibilities

- Serve agent schemas via /agents/:agentId/schema
- Validate incoming task input
- Reject invalid tasks before sending to Redis

Frontend Responsibilities

- Fetch schema
- Render dynamic input forms
- Validate client-side before sending to Gateway

Agent Harness Responsibilities

- Validate input before execution
- Use schema metadata for runtime configuration

Workflow Engine Responsibilities

- Use schema to configure workflow nodes
- Validate node input before execution

Summary

This Agent Schema Loader provides:

- Unified schema loading
- Input validation
- Schema caching
- Gateway + Frontend + Agent Harness integration
- Future-proof extensibility for multi-agent systems

It completes the final missing piece of your MVP architecture.